The First Reply

Last time ELIZA learned to see every keyword in a sentence. To answer, it needs exactly one – and a reply for it.

Marcus Birkenkrahe

COSC 1210 · CS I · Session 2 of 2 · Augustana University

Fall 2026 · See also: Downey, *Think Python* 3e, ch. 7

Review: mini challenge (10 min)

→ Switch to editor · mini challenge · 10 min, then we discuss

```
def count_keywords(sentence):
    count = 0
    for line in open("keywords.txt"):
        word = line.strip()
        if word in sentence.lower():
            count += 1
    return count
```

1. Give count_keywords a second parameter: `count_keywords(sentence, filename)`. Make a second keyword file:

```
open("feelings.txt", "w").write(
    "happy\nproud\ncalm\nexcited\n")
```

sentence	filename	returns
"I am happy and proud."	"feelings.txt"	2
"Sorry, I am so excited!"	"feelings.txt"	1
"My mother and my father love to talk."	"keywords.txt"	2

2. Write a second function `count_keywords_nostrip(sentence, filename)`: the same, without the `.strip()` line, testing `line` instead of `word`. What does it return for the last sentence – and why?

Review: mini challenge – solution

1. Any keyword file

```
def count_keywords(sentence, filename):
    count = 0
    for line in open(filename):
        word = line.strip()
        if word in sentence.lower():
            count += 1
    return count
```

```
2
1
2
```

2. Without `.strip()`

```
def count_keywords_nostrip(sentence, filename):
    count = 0
    for line in open(filename):
        if line in sentence.lower():
            count += 1
    return count

print(count_keywords_nostrip(
    "My mother and my father love to talk.",
    "keywords.txt"))
```

```
0
```

Review: mini challenge – discussion

- Two changes make it work for any keyword file: the new parameter and `open(filename)`.
- Without `.strip()` the result is 0: every line ends with `\n`, and `'mother\n'` is not in the sentence.
- `"Sorry, sorry, sorry!"` counts 1, not 3: the loop goes over the keywords, not over the sentence.
- An empty last line in the file adds 1: after `.strip()` it is `""`, and `""` is in every string.

Search: stop at the first keyword

→ **Switch to editor · code-along 1**

```
def find_keyword(sentence):
    for line in open("keywords.txt"):
        word = line.strip()
        if word in sentence.lower():
            return word
    return None

print(find_keyword("I had a strange dream."))
print(find_keyword("The weather is nice."))
```

```
dream
None
```

birkenkrahe.github.io/go/elizakeys

A **linear search**: look at the keywords one by one, stop at the first match.

- `return word` ends the search – the rest of the file is not read
- `return None` runs only if nothing matched: "not found" has read the whole file – the slowest case
- Writing `return None` says that `None` is a planned answer

Several keywords: which one wins?

→ **Switch to editor · TRY (2 min) · predict, then run**

```
print(find_keyword("My mother says I always dream about computers."))
```

```
dream
```

The sentence mentions “mother” first. Why “dream”?

Question: which keyword wins?

Line	Keyword
1	sorry
2	remember
3	**dream**
4	computer
5	mother
...	...

- The loop goes through the **file**, not through the sentence
- "dream" is line 3, "mother" line 5: the order of the file decides
- ELIZA gave each keyword a **rank**: the highest-ranked keyword wins. You build that in chapter 11

Is there any keyword? A Boolean function

→ **Switch to editor · code-along 2**

```
def has_keyword(sentence):
    for line in open("keywords.txt"):
        if line.strip() in sentence.lower():
            return True
    return False
```

The same loop as in `find_keyword`. Don't write it twice.

Reuse the search:

```
def has_keyword(sentence):
    return find_keyword(sentence) != None

print(has_keyword("I had a strange dream."))
print(has_keyword("The weather is nice."))
```

```
True
False
```

One line, no `if`. (Usually written `is not None` – `is` comes in chapter 9.)

Use a Boolean function as the condition

```
sentence = "I always forget my keys."
if has_keyword(sentence):
    print("ELIZA has a rule for:", find_keyword(sentence))
```

```
ELIZA has a rule for: always
```

No == `True`: `has_keyword(sentence)` already *is* `True` or `False`.

The first reply

→ **Switch to editor · code-along 3**

```
def respond(sentence):
    keyword = find_keyword(sentence)
    if keyword == "sorry":
        return "Please don't apologize."
    elif keyword == "dream":
        return "What does that dream suggest to you?"
    elif keyword == "computer":
        return "Do computers worry you?"
    elif keyword == "always":
        return "Can you think of a specific example?"
    else:
        return "Please go on."

print(respond("Sorry, I am late."))
print(respond("I always forget my keys."))
print(respond("The weather is nice."))
print(respond("My brother never calls."))
```

```
Please don't apologize.
Can you think of a specific example?
Please go on.
Please go on.
```

- The `else` gives the **fallback reply** – and makes sure every path returns
- "brother" is a keyword, but no branch handles it: that is today's practice

What does ELIZA say?

→ Switch to editor · TRY (2 min) · predict, then run

```
print(respond("I dreamed about my computer."))
```

```
What does that dream suggest to you?
```

→ “dream” is inside “dreamed” (substring), and “dream” comes before “computer” in the file.

Tests in the docstring

→ **Switch to editor · code-along 4**

```
def find_keyword(sentence):
    """Return the first keyword in sentence, or None.

    >>> find_keyword("I had a strange dream.")
    'dream'
    >>> print(find_keyword("The weather is nice."))
    None
    """
    for line in open("keywords.txt"):
        word = line.strip()
        if word in sentence.lower():
            return word
    return None
```

- A **doctest**: >>> is a call, the next line the **expected output**
- A returned string shows with quotes: `'dream'`
- None shows as nothing – so the test prints it

Run the tests

```
from doctest import run_docstring_examples

def run_doctests(func):
    run_docstring_examples(func, globals(),
                           name=func.__name__)

run_doctests(find_keyword)
```

No output: all examples passed.

Now add an example with a wrong expectation and run again:

```
>>> find_keyword("My computers are slow.")
'computers'
```

```
**********************************************
File "__main__", line 6, in find_keyword
Failed example:
    find_keyword("My computers are slow.")
Expected:
    'computers'
Got:
    'computer'
```

The call, what you expected, what came back. Here the *test* was wrong: the keyword is “computer”. Fix it, run again – no output.

Practice: a family rule

→ **Switch to editor · 10 min, then 5 min discussion**

1. Write a Boolean function `is_family(keyword)`: `True` for "mother", "father", "sister" or "brother", else `False`. One `return`, no `if`.
2. Add a branch to `respond`: for a family keyword, ELIZA says "Tell me more about your family."

```
>>> print(is_family("mother"))
True
>>> print(is_family("dream"))
False
>>> print(respond("My brother never calls."))
Tell me more about your family.
```

Checkout

1. Why is "not found" the slowest case of a linear search?

→ The search can only say "not found" after it has looked at every keyword. A match stops it early.

2. What does a doctest print when all its examples pass?

→ Nothing.

Summary

- A **linear search** looks at the items one by one and stops at the first match. "Not found" (`return None`) goes after the loop and means every item was checked
- The order of the keyword file decides which keyword wins when a sentence has several
- A Boolean function can reuse another function: `return find_keyword(sentence) != None`. Use it in `if` without `== True`
- ELIZA's reply comes from an `if` / `elif` chain; the `else` gives the **fallback reply** and makes sure every path returns
- A **doctest** puts calls (>>>) and their expected output into the docstring. `run_docstring_examples` runs them; no output means all passed

Glossary: terms from session 2

#	Term	Meaning
1	search	Look through items for one with a property
2	linear search	Check items one by one, stop at the first hit
3	fallback reply	Reply used when no keyword matches
4	rank	Priority of a keyword when several match
5	doctest	Test written as >>> examples in a docstring
6	expected output	What a test says a call must print or return

Next: strings

ELIZA's replies are fixed sentences. The real ELIZA reuses your words:

"I am tired" $\rightarrow$ "Why are you tired?"

- Take strings apart with indexes and slices
- ELIZA writes things down – and the setup cell that made `keywords.txt` finally makes sense